

第 02 讲：基座模型能力 vs Harness 能力

Coding Agent Notes (June 2026)

1 核心图景

本讲的核心问题是：一个 coding-agent benchmark 分数到底来自模型，还是来自 harness？直觉上我们会说“模型 A 比模型 B 强”，但在 repo-level task 里，观测到的分数往往不是模型能力的纯读数，而是整个系统共同作用的结果。



Observed score = model capability × harness design × repo context
× action space × validation policy × budget × task distribution.

这里的关键不是否认模型能力。没有足够强的基座模型，任何 harness 都只能把弱能力包装得更复杂。但在长期编码任务中，模型的能力必须经过一层工作制度才能显现：模型怎样看文件、怎样编辑、怎样运行测试、怎样接收反馈、怎样停止、怎样选择候选 patch。

因此，看到任何 SWE-style 分数时，都应该先问：这是模型本身的差距，还是系统给模型提供了更好的行动接口、更好的上下文、更强的验证、更大的 rollout budget？

1.1 为什么这个区分重要

比较模型时需要公平边界。 如果两个结果使用不同检索器、不同编辑工具、不同重试次数和不同验证策略，那么 pass rate 不能直接解释为模型能力差距。

复现结果时需要知道系统变量。 SWE-bench 这类任务要跑真实仓库测试，任何环境、依赖、patch 应用方式、候选选择策略都会改变结果。

部署产品时需要知道责任归属。 如果 agent 失败，是模型没有理解 issue，还是 harness 没给到相关文件，还是工具反馈太长，还是停止规则太早？不同原因对应完全不同的修复方式。

Insight. benchmark 分数是系统测量值，不是模型体检报告。把它当作纯模型能力，会误判研究进展，也会在产品部署时修错层。

1.2 一个最小例子

设想同一个 base model 面对同一个 repo issue。第一种设置只给它自然语言 issue 和自由 shell；第二种设置给它结构化搜索、稳定编辑命令和短测试反馈；第三种设置再加入多候选 patch 生成和自动 validation。三种设置的模型权重完全相同，但最终 pass rate 可能明显不同。

这并不矛盾。第一种设置主要考验模型能否自己在复杂环境里找到行动路径；第二种设置把模型能力更稳定地转化成可执行动作；第三种设置又把一次生成变成“生成多个候选，再用验证筛选”。如果只看最后分数，就会把三种完全不同的系统行为压缩成一个数字。

这个例子说明：模型能力是必要条件，但不是观测分数的充分解释。观测分数至少包含三层信息：模型本身能否理解和生成；harness 是否让它稳定行动；验证和预算是否给它足够机会纠错。

2 模型能力是什么

模型能力是 agent 的底层认知和生成能力。它包括理解自然语言 issue、阅读代码、推理依赖关系、生成 patch、解释测试失败、规划下一步动作，以及遵守指令和格式。

可以把模型能力分成四类：

能力	作用	不足时的表现
语义理解	读懂 issue、测试失败、代码意图	修错问题或忽略约束
代码推理	理解跨函数、跨文件依赖	patch 位置合理但逻辑错误
代码生成	产生可应用 diff 或编辑	语法错、API 用错、风格不一致
策略判断	决定先定位、先测试还是先改	乱试、过早停止、无效探索

这些能力是真实存在的。一个弱模型即使拿到好 harness，也可能无法理解错误信息，或者写出无法通过基本测试的 patch。但模型能力只能解释“它有可能做什么”，不能完全解释“它最终做成了什么”。

原因是 coding agent 的行动要经过外部接口。模型可能知道应该修改某个函数，却不会稳定地在 shell 里定位文件；模型可能理解测试失败，却被过长的 stdout 淹没；模型可能写出正确 diff，却因为工具格式错误没有应用成功。这些不是模型语义能力本身的问题，而是模型能力和 harness 之间的接口问题。

2.1 模型能力的上限和下限

模型能力决定系统上限的一部分。弱模型常见的问题不是工具层能修好的：它可能看不懂 issue，误解测试失败，写出不存在的 API，或者把异常处理改成吞掉错误。这样的失败即使放进更好的 harness，也只能被更快暴露，不能自动变成正确 patch。

但模型能力也有被低估的时候。强模型如果面对极差的 observation format，可能无法从几千行日志里找到关键错误；如果编辑接口要求脆弱的手写 diff，它可能在格式上失败；如果工具输出没有稳定状态标记，它可能不知道某个修改是否真的应用成功。此时低分不是模型“不会修”，而是模型无法通过当前接口把能力转化成动作。

现象	更像模型瓶颈	更像接口瓶颈
误解 issue	把需求修成另一件事	issue 被截断或上下文缺失
写错 API	不知道库语义	没看到 import 或调用样例
无法定位	不会读调用链	搜索工具太弱或 repo map 缺失
patch 无法应用	diff 构造能力差	编辑工具格式太脆
反复试错	规划和停止差	反馈没有压缩成信号

因此，模型能力不能用一句“强”或“弱”概括。更好的说法是：这个模型在某种 harness、某种上下文选择、某种动作语言和某种验证预算下，能把多少认知能力转化成可验证修复。

3 Harness 能力是什么

Harness 是模型和软件环境之间的工作制度。它决定模型看什么、能做什么、怎么得到反馈、怎样把失败带回下一轮。

Harness 变量	控制什么	典型失败
上下文选择	哪些文件、符号、历史进入窗口	漏关键文件、上下文污染
动作接口	搜索、编辑、运行命令的格式	malformed tool call、编辑失败
观察格式	stdout、diff、测试失败怎样返回	反馈太长、信号不清
验证策略	何时跑测试、怎样筛 patch	过拟合局部测试
停止规则	何时提交、重试、回滚或升级	过早停止或无限探索
预算设置	token、时间、候选数、并行 rollout	分数不可比较

SWE-agent 把其中一部分明确命名为 agent-computer interface (ACI): agent 使用的命令和计算机返回的反馈格式。这个命名很重要，因为它把 LM agent 当成一种新的 end user。人类工程师适合用 shell、IDE、浏览器；LM 未必适合原封不动地使用人类界面。

OpenHands 把 harness 提升到平台层: event stream、docker sandbox、bash、browser、IPython server、multi-agent delegation、evaluation 都成为系统的一部分。到了这个层次，harness 不只是 prompt 或工具列表，而是 agent 的运行环境。

Insight. Harness 的作用不是“帮模型作弊”，而是定义模型如何接触现实。界面设计不好时，强模型也会像新手一样在文件系统、编辑器和测试反馈之间迷路。

3.1 Harness 是控制平面

可以把 harness 看成 coding agent 的控制平面。它不直接“懂代码”，但它决定模型怎样使用代码环境：何时搜索、如何打开文件、怎样提交编辑、如何运行测试、如何截断日志、失败后是否重试、何时停止。控制平面设计得好，模型会在更小、更清晰、更可恢复的动作空间中工作；设计得差，模型会把大量能力消耗在与环境摩擦上。

控制平面最重要的不是功能数量，而是可观察性和可恢复性。一个工具如果失败了，模型和系统都应该知道失败发生在哪里；一个测试如果超时了，系统应该区分是代码错误、环境错误还是预算错误；一个编辑如果没有应用成功，轨迹里应该能看出来。否则失败会被误写进上下文，后续动作会建立在错误状态上。

3.2 Harness 不是作弊

在模型评测语境里，强 harness 容易被误解成“给模型作弊”。这个说法只在一种情况下成立：harness 引入了测试答案、未来 commit、人工筛选或 benchmark-specific 泄漏。除此之外，良好的工具接口、上下文选择和验证策略并不是作弊，而是软件工程系统本来就需要的工作制度。

人类工程师也依赖 IDE、grep、LSP、测试、debugger、CI 和 code review。coding agent 的 harness 只是把这些制度重新设计成模型可用的形式。真正要警惕的不是 harness 本身，而是报告是否清楚说明了 harness 的边界、预算和验证方式。

4 同一个模型，不同 harness

最容易看出 harness 作用的方法，是固定模型，改变系统外壳。



4.1 SWE-agent: 接口改变能力上限

SWE-agent 的核心贡献不是“多给模型几个命令”，而是说明 ACI 会显著影响 agent 的行动质量。它不让模型直接面对过于自由、过于细粒度的 raw shell，而是提供更适合 LM 的查看、搜索、编辑和测试反馈。

在论文报告中，SWE-agent with GPT-4 Turbo 在 full SWE-bench 上达到 12.47% pass@1，明显高于 SWE-bench 原论文中的早期非交互式 baseline。这里不能简单解读为“GPT-4 Turbo 变强了”；更准确地说，是同一类基础模型能力通过更合适的行动接口被释放出来。

4.2 Agentless: 少自治也可能更强

Agentless 提供了另一个方向。它不让模型在复杂工具空间中自由决定下一步，而是把任务拆成 localization、repair、patch validation 三阶段。模型仍然参与每个子任务，但全局流程由外部 pipeline 控制。

这说明 autonomy level 本身就是 harness 变量。更多自治不一定更强，因为复杂行动空间会放大错误：定位错会影响修复，修复候选太多会增加验证成本，错误反馈会污染后续决策。一个结构化流程有时能用更少自由度换来更高可调试性。

4.3 CodeAct: 动作语言改变可组合性

CodeAct 进一步说明，动作表示也属于 harness。JSON/tool-call action 往往清晰、可控，但组合性有限；executable code action 允许模型调用库、组合操作，并根据执行错误继续调整。动作空间变了，同一个模型能表达的计划也会变。

这不是说 code-as-action 永远更好。更强的动作空间也带来更高风险：更难约束、更难审计、更容易执行副作用。因此，动作语言既是能力问题，也是安全和可控性问题。

Insight. 同一模型在 raw shell、ACI、structured pipeline、code-as-action 中表现不同，不是矛盾，而是 coding-agent 评测的本质：我们测到的是模型通过某种接口完成任务的能力。

5 结果怎么读

读 coding-agent 结果时，至少要问六个问题。



问题	要看的内容	为什么重要
Model?	base model、版本、上下文长度	决定基础理解和生成能力
Harness?	prompt、loop、工具、停止规则	决定行动制度
Context?	检索器、repo map、历史压缩	决定模型看到什么
Validation?	tests、patch selection、oracle	决定反馈是否可靠
Budget?	token、时间、候选数、rollout	决定探索空间
Task split?	full、Lite、Verified、自建集	决定结果可比性

一个常见误读是只看 leaderboard 排名。更好的读法是把结果拆开：这个系统提高了哪一层？如果是模型更强，应该在相同 harness 下比较；如果是 harness 更强，应该说明 interface、context、validation 或 budget 的变化；如果两者都变了，就不要把结论写成单纯的模型比较。

这也是为什么很多论文会做 ablation。Ablation 的目的不是凑表格，而是帮助归因：去掉某个工具、换掉某种观察格式、减少候选 patch、取消 validation 后，分数怎样变化。没有归因，分数只是现象。

5.1 观测分数里的混杂变量

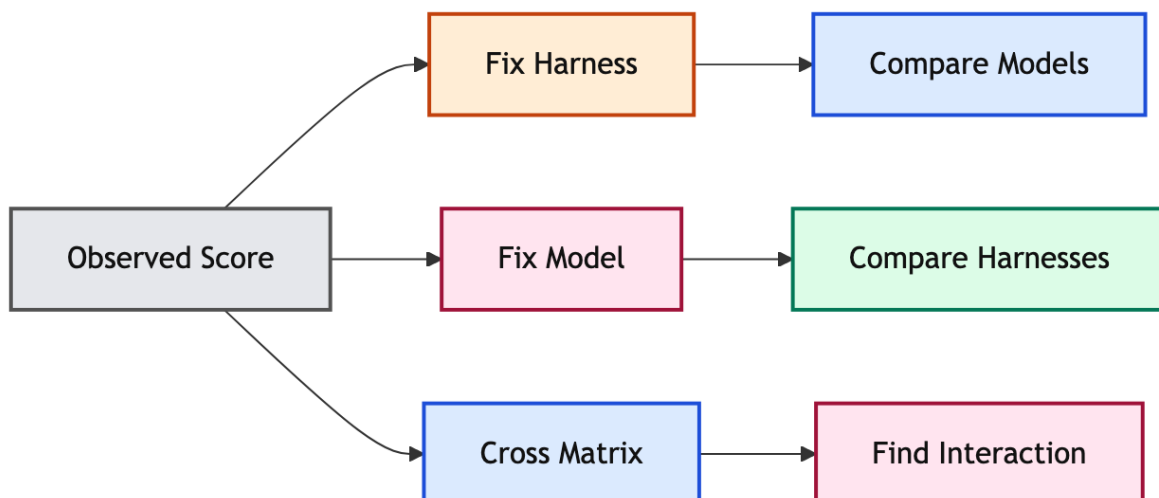
一个结果表里通常会有 pass rate、模型名和 benchmark split，但真正决定可比性的变量更多。下面这些变量如果没有写清楚，分数就很难被解释。

变量	可能带来的提升	容易造成的误读
上下文检索	找到关键文件和测试	当成模型定位能力
动作语言	更容易组合工具和编辑	当成模型规划能力
验证策略	筛掉坏 patch	当成一次生成质量
重试预算	扩大搜索空间	当成单次决策更强
停止规则	减少无效轨迹	当成模型更会判断完成
任务过滤	去掉不可复现实例	当成泛化能力提高

这些变量不一定是坏事。好的检索、动作语言和验证策略本来就是 agent 系统的一部分。问题在于归因：如果论文或产品报告没有区分它们，就不能把结果直接说成模型能力提升。

6 怎样做公平归因

如果目标只是发布一个可用 agent，那么“系统 A 比系统 B 分数高”已经有价值。但如果目标是理解研究进展，就必须进一步问：提高来自模型、harness，还是两者的交互？



实验设计	固定什么	能回答什么
固定 harness, 比模型	prompt、工具、预算、验证	哪个模型在同一制度下更强
固定模型, 比 harness	base model、任务集、预算	哪个接口更释放能力
模型 × harness 矩阵	多模型、多 harness 交叉	是否存在适配或交互项
固定任务切分	split、过滤规则、环境	分数是否真的可比较

固定 harness 比模型 是最接近“模型评测”的设计。比如相同 task split、相同 context pipeline、相同工具 schema、相同最大步数，只替换 base model。此时分数变化更能说明模型理解、推理和生成能力的差异。但这个结论仍然只在该 harness 内成立：一个模型在 structured pipeline 下强，不代表它在自由 shell-loop 里同样强。

固定模型比 harness 是最接近“系统设计评测”的设计。比如同一个模型分别放进 raw shell、SWE-agent ACI、Agentless pipeline、CodeAct action space。此时分数变化主要反映行动接口、上下文、验证和停止策略的作用。这类实验能告诉我们：能力瓶颈是不是出在模型和环境之间的接口层。

交叉矩阵 最有解释力。因为模型和 harness 常常不是相加关系，而是交互关系：复杂工具可能帮助强模型，却让弱模型更容易迷路；结构化 pipeline 可能限制强模型的探索，但显著减少弱模型的错误传播；长上下文可能让某些模型利用更多证据，也可能让另一些模型被噪声干扰。

Insight. 只要模型和 harness 同时变化，结论就应该写成“系统 A 优于系统 B”。只有控制变量之后，才可以进一步说“模型更强”或“harness 更好”。

6.1 三类实验分别回答什么

固定 harness, 比模型。 这是最接近 model evaluation 的实验。它要求 task split、prompt 模板、工具 schema、上下文检索、最大步数、验证脚本和预算都保持一致，只替换 base model。这样的实验能回答：在同一套制度下，哪个模型更会理解 issue、读代码、生成 patch 和利用反馈。

它的局限是外推性。某个模型在结构化 pipeline 下更强，不代表它在自由工具空间中也更强；某个模型在短观察格式下表现好，不代表它能处理长日志；某个模型适合 code-as-action，也不代表它适合 JSON 工具调用。

固定模型, 比 harness。 这是最接近 system-design evaluation 的实验。它回答：同一个模型放进不同界面、不同 context selector、不同 validation policy、不同 action space 后，能力释放程度怎样变化。SWE-agent 和 Agentless 这类工作最有价值的部分，正是在这一层提供系统设计证据。

它的局限是模型相关性。某个 harness 可能特别适配 GPT-4 系列，也可能特别照顾弱模型；换成另一个模型后，复杂工具、长上下文或强约束 pipeline 的效果可能改变。

模型 × harness 矩阵。 这是最接近 causal diagnosis 的实验。它不只问“谁更强”，而是问“提升是否依赖某个组合”。如果 harness A 只提升强模型，对弱模型无效，那么它可能要求更高的工具理解能力；如果 harness B 对弱模型帮助大、对强模型帮助小，那么它可能主要减少错误空间。

6.2 负结果也有信息

归因实验不应该只展示能提高分数的组件。没有提升甚至降低分数的 ablation 也很重要。例如，增加更自由的 shell action 可能让强模型更灵活，却让弱模型更容易误操作；增加更多上下文可能给模型更多证据，也可能引入噪声；增加验证轮次可能提高 pass rate，也可能过拟合可见测试。

这些负结果能帮助读者理解机制边界。一个系统组件真正有价值，不是因为它在某个表格里涨分，而是因为我们知道它在什么条件下有效、在什么条件下无效、失败时会造成什么副作用。

7 常见错误归因

把预算当能力。

一个 agent 可以跑更多候选 patch、更多验证轮次、更多 self-reflection，再从中挑最好的结果。分数提高可能来自探索空间扩大，而不是单次决策更强。因此 pass@1、best-of- n 、parallel rollout、人工筛选和自动筛选必须分开写。

把检索当理解。

如果系统有很强的 repo map、embedding retrieval、symbol search 或 handcrafted localization，它可能已经把关键文件送到模型面前。模型确实还要修复代码，但“找到问题位置”的能力不应全部记到模型头上。

把验证当修复。

Patch validation 可以过滤大量看似合理但跑不通的候选。Agentless 的价值之一就在于把 validation 独立成阶段。此时最终提交的 patch 更可靠，但这不等价于模型第一次生成 patch 的质量更高。

把产品 harness 当 benchmark harness。

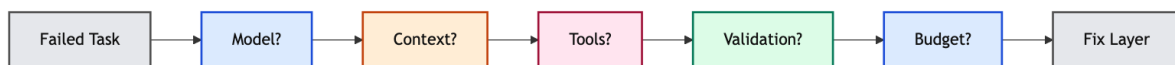
Benchmark harness 通常有固定数据集、固定容器、固定测试脚本和明确终止条件。真实产品 harness 还要处理权限、用户上下文、未提交代码、部分测试不可运行、长任务恢复、人工确认和代码评审。两者相似，但不能默认等价。

把短任务结论迁移到长任务。

在短任务里，模型一次性读题、写答案即可；在 repo issue 里，agent 要经历定位、编辑、运行、失败、回退、再定位。越长的任务越依赖 harness 的记忆、压缩、验证和恢复机制。

8 失败归因流程

当一个 coding agent 没有解决任务时，最差的排查方式是直接说“模型不够强”。模型可能确实是瓶颈，但如果不沿着系统层拆开，下一步改进就会变成盲目换模型、加 token、加工具或加 retry。



8.1 先看任务是否被理解

第一步是判断模型是否理解了 issue 和成功条件。如果 agent 从一开始就修错目标，例如把性能问题当成语法 bug，或者把用户请求改成相反行为，那么主要瓶颈可能在模型语义理解、prompt 指令或 issue context。此时增加工具预算通常没有用，因为系统会更努力地修错问题。

8.2 再看上下文是否足够

如果目标理解正确，但 patch 位置错误，下一步要看 context。关键测试是否进入上下文？相关实现文件是否被找到？调用链是否展开？历史失败是否被保留？很多 repo-level 失败不是模型不会写 patch，而是系统没有把模型带到正确代码区域。

8.3 再看动作和观察是否可靠

如果模型找到了正确区域但编辑失败，要检查工具层：diff 是否应用成功，文件是否真的修改，命令是否在正确 cwd 下执行，测试输出是否被截断。长期任务中，工具失败如果没有被显式记录，会伪装成模型失败。

8.4 最后看验证和预算

如果 patch 看起来合理但最终不通过，要看验证策略和预算。是否只跑了局部测试？是否因为时间限制跳过了关键验证？是否生成了多个候选但选择策略错误？是否过早停止？这些问题都属于 harness 和 evaluation 层，而不是纯模型层。

排查阶段	证据	优先修什么
目标理解	轨迹是否修对问题	prompt、issue context、模型
上下文选择	是否看到关键文件/测试	retrieval、repo map、搜索工具
动作执行	编辑和命令是否生效	tool schema、runtime、cwd
观察压缩	是否读懂测试失败	log summarization、反馈格式
验证预算	是否有足够尝试和筛选	rollout、validation、停止规则

Insight. 失败归因的目标不是给模型或 harness “分锅”，而是找到最便宜、最有效的改进层。模型错了就换模型或训练；context 漏了就改检索；工具不稳就改接口；验证不足就改测试和预算。

9 对系统设计的启发

不要把 benchmark harness 和产品 harness 混在一起。如果研究结果在一个高度调优的 benchmark harness 中得到，部署到产品时却换成不同工具、权限、UI 和反馈格式，能力会重新分布。

把 harness 变量写成显式配置。模型、prompt、工具 schema、上下文检索、验证脚本、重试预算、停止规则都应该可记录、可复现。否则失败时很难知道该改模型还是改系统。

训练数据也要区分模型行为和 harness 行为。如果训练轨迹来自某个特定 harness，模型学到的不只是“如何修 bug”，还包括“如何在这个接口里修 bug”。换接口后，行为可能迁移不完全。

评测报告应该展示失败类型。只报告 pass rate 不够。应该区分定位失败、编辑失败、测试误读、工具失败、环境失败和停止失败。不同失败类型对应不同改进路径。

失败类型	更像模型问题	更像 harness 问题
定位失败	误解 issue 语义	没拿到关键文件
编辑失败	API 或逻辑错误	diff 工具不稳定
测试误读	看不懂失败原因	输出过长或截断
停止失败	不会判断完成	终止规则设计差

这个表不是为了把责任机械切开，而是给排错一个入口。真实失败往往是混合原因：模型误解了 issue，可能是因为 context selector 漏掉了测试；模型写了错误 patch，可能是因为 validation 只跑了过窄的测试；模型反复乱试，可能是因为 observation format 没把关键错误压缩出来。

10 从研究结果到产品决策

研究报告里的 harness 和产品里的 harness 通常不一样。研究环境追求可控、可复现、可比较；产品环境还要处理用户上下文、权限、未提交代码、隐私、UI 状态、人工确认和回滚。把研究结果迁移到产品时，不能只看 pass rate。

第一，产品更关心可恢复性。Benchmark 任务失败了可以记为 0 分；产品任务失败后，用户还要继

续工作。系统必须能解释做过什么、改过什么、哪些测试失败、能否回滚。一个高分但轨迹不可审计的 agent，很难放进真实开发流程。

第二，产品更关心权限边界。 Research harness 可以在 sandbox 里自由运行命令；产品 agent 可能接触用户仓库、私有依赖、环境变量、浏览器 session 和部署权限。工具越强，越需要确认、隔离和审计。

第三，产品更关心交互成本。 人类在环可以提升安全性，但如果每一步都要求确认，agent 就会变成打扰源。好的 harness 要把高风险动作交给人类确认，把低风险、可恢复动作自动化，把失败摘要做得足够短。

维度	研究 harness	产品 harness
目标	可比较 pass rate	可用、可恢复、可审计
环境	固定容器和测试	用户仓库、权限、未提交状态
反馈	测试结果为主	测试、UI、review、人类确认
风险	benchmark leakage、过拟合	数据泄漏、破坏性动作、误合并
成功	patch 通过 oracle	用户接受且可维护

Insight. 研究 harness 告诉我们系统在受控任务里的能力；产品 harness 决定这些能力能否被安全地交到用户手里。把两者混为一谈，会高估可部署性。

11 一份结果报告至少要写什么

如果一份 coding-agent 结果只写“模型 X 在 benchmark Y 上达到 Z%”，读者很难判断这个数字的含义。最低限度的报告应该让读者能复现系统边界，并能区分模型贡献和 harness 贡献。

报告项	应该写清楚什么	不写会怎样误读
Model	名称、版本、上下文长度、是否微调	把模型差异和系统差异混在一起
Harness	loop、prompt、工具 schema、停止规则	不知道分数来自哪套工作制度
Context	检索器、repo map、压缩方式	把检索能力当成模型理解能力
Action	shell、编辑器、code action、browser	不知道模型实际能做什么
Validation	测试集合、候选筛选、oracle 边界	把验证筛选当成生成质量
Budget	token、时间、重试、并行 rollout	把搜索规模当成单次能力
Failures	定位、编辑、测试、环境、停止失败	只知道没过，不知道该改哪层

这张表不是为了让报告变长，而是为了让结果可解释。模型研究者需要知道是不是 base model 的能力提升；系统研究者需要知道是不是 interface、context 或 validation 起作用；产品工程师需要知道失败后应该换模型、改工具、加验证，还是降低权限和预算。

11.1 一个更诚实的表述方式

如果模型、harness、context、validation 和 budget 都变了，最诚实的表述是：

System A 在给定任务分布、预算和验证策略下优于 **System B**。
这个结果不能单独归因于 base model，除非其他系统变量保持不变。

只有当控制变量成立时，才应该把结论写成“模型 A 更强”或“harness B 更好”。如果只是更换了模型，就说模型比较；如果只是更换了 harness，就说系统接口比较；如果两者都换了，就说系统比较。这个措辞看似保守，但它能防止研究结论被过度营销，也能防止工程团队修错层。

11.2 为什么失败类型比平均分更有用

平均 pass rate 告诉我们总体成功率，但失败类型告诉我们下一步该做什么。定位失败多，应该改 context 和 repo navigation；编辑失败多，应该改 action interface；测试误读多，应该改 observation compression；环境失败多，应该改 runtime；停止失败多，应该改 loop policy。

因此，一个 28% pass rate 但失败类型清楚的系统，可能比一个 32% pass rate 但轨迹不可解释的系统更有研究价值。前者告诉我们如何继续改进；后者只告诉我们它在某个设置下赢了。

Insight. 好的结果报告不是把分数包装得更漂亮，而是把分数拆到可行动的系统层。能指导下一步改进的分数，才是有工程价值的分数。

12 局限与开放问题

模型和 harness 不能完全分离。 强模型可能更会利用复杂工具，弱模型可能需要更强约束。某个 harness 对 GPT-4 系列有效，不代表对所有开源模型都有效。

强 harness 可能掩盖弱模型。 如果外部 pipeline 做了大量定位、候选生成、重排和验证，最终分数可能高，但模型本身的长期决策能力并没有同等提高。

弱 harness 可能低估强模型。 如果接口过于自由、反馈太长、编辑工具不稳定，强模型也可能因为行动失败而得低分。

Harness 可能过拟合 benchmark。 如果搜索策略、验证脚本、候选选择只针对某个 split 优化，分数可能高于真实可部署能力。

13 最终 takeaway

coding-agent 分数是系统结果，不是纯模型结果。

模型能力决定上限的一部分；harness 决定能力能否被稳定转化为行动、观察、修复和验证。

比较任何 agent，都要先问清楚：模型是谁，harness 是什么，context 怎么来，validation 怎样做，budget 有多大。

参考资料

- [1] [SWE-agent](#). agent-computer interface
- [2] [Agentless](#). structured repair pipeline
- [3] [SWE-bench](#). repo-level benchmark
- [4] [OpenHands](#). software-agent platform
- [5] [CodeAct](#). executable action space